API Lifecycle DevOps Project Report

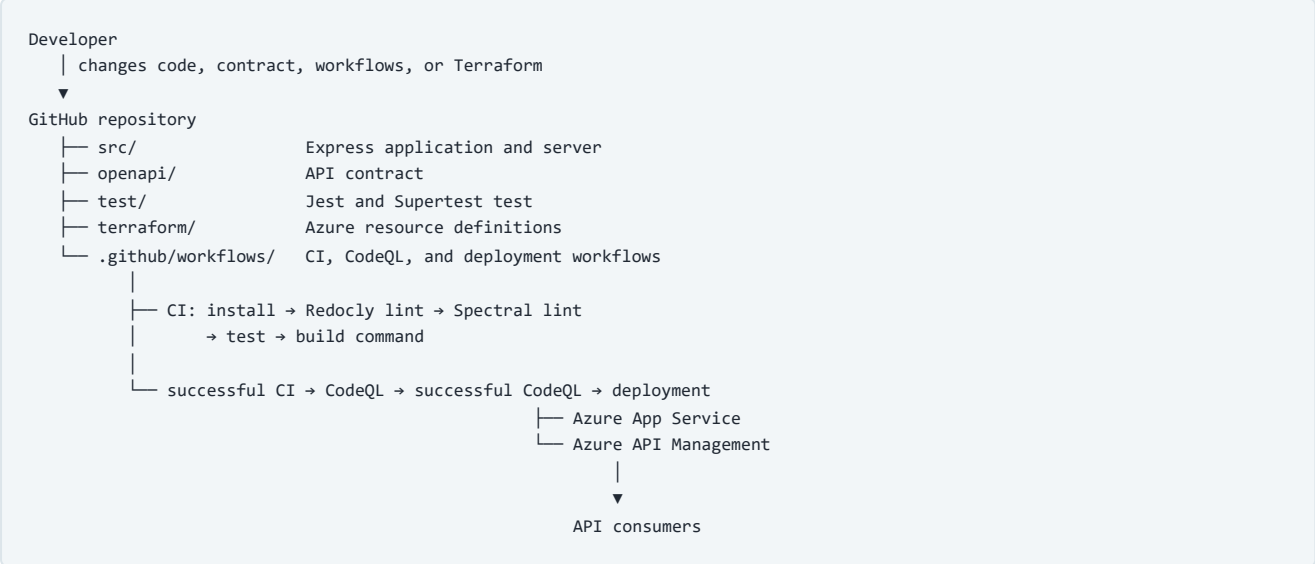
1. Purpose and scope

This project demonstrates an API lifecycle workflow built around a small weather REST API. Its main purpose is to show how an API contract, application code, automated quality checks, security analysis, cloud deployment, and API publication can fit together in one repository.

The API is implemented with Node.js and Express. Its `GET /weather/{city}` operation returns the requested city with a fixed temperature and condition. The fixed response keeps the application simple so that the demonstration can focus on the delivery process rather than a live weather provider or data store.

The report distinguishes between behavior present in the repository and capabilities that would need further implementation. In particular, Terraform defines Azure resources, but neither GitHub Actions workflow runs Terraform.

2. Architecture



The design separates the API contract, application implementation, infrastructure declarations, and automation. Git provides a shared, version-controlled location for these components; GitHub Actions coordinates the quality and delivery steps; Azure App Service hosts the Node application; and Azure API Management (APIM) exposes an imported version of the OpenAPI definition.

3. Components and their roles

API contract

`openapi/openweather-api.yaml` describes the API using OpenAPI 3.0.3. It defines the weather path, required city path parameter, success response schema, and operation metadata. The contract is checked by both Redocly CLI and Spectral, giving the project a machine-readable interface that can be linted and imported into APIM.

Application

`src/app.js` defines the Express route `GET /weather/:city`, returning JSON with the city parameter and fixed sample values. `src/server.js` starts the app on the platform-provided `PORT`, defaulting to port 3000 for local use. Separating the Express app from the listener allows the test to import the app directly without starting a network server.

Automated checks

The CI workflow uses Node.js 20 and installs the lockfile-pinned dependencies with `npm ci`. It then runs Redocly lint, Spectral lint, the Jest test suite, and the package's build script. The current test uses Supertest to call the route and asserts its status code and response body.

The build script currently prints “Build Successful”; it does not compile, bundle, or package the application. The application is plain JavaScript, so this step is a pipeline placeholder rather than a substantive build.

Security analysis

The CodeQL workflow analyzes JavaScript after a successful completion of the CI workflow. It grants the permissions needed to read repository contents and publish security-analysis results. This demonstrates static analysis in the delivery process, but it does not by itself establish that the application is secure or replace dependency, runtime, or manual security testing.

Azure infrastructure and API gateway

The Terraform configuration declares a resource group, an Azure API Management service, a Linux App Service plan, and a Linux Web App configured for Node.js 20. It also declares outputs for the resource group, APIM, gateway URL, and web app URL.

Terraform is currently a separate infrastructure definition: no checked-in GitHub workflow runs `terraform plan` or `terraform apply`. Provisioning must therefore be performed separately. The deployment workflow assumes the Azure resources already exist.

The APIM deployment step imports the OpenAPI file as an API and configures its backend URL to point to the App Service. This connects the published gateway API to the deployed application. Azure login uses the `AZURE_CREDENTIALS` GitHub secret; the Web App deployment also uses `AZURE_WEBAPP_NAME`.

4. Processes and features demonstrated

Continuous integration

The CI workflow runs on pushes to the `main` branch and can also be started manually with `workflow_dispatch`. It does not currently declare a pull-request trigger. On each run, the workflow installs dependencies and gates completion on the specification checks, test, and build command. A failing command fails the job and prevents the successful-CI path from proceeding.

This demonstrates repeatable validation and feedback on changes to the main branch. Redocly checks the OpenAPI definition, while Spectral applies the configured OpenAPI style rules. Jest and Supertest verify the implemented route response.

Security analysis and deployment sequence

CodeQL is triggered when the named CI workflow completes and proceeds only when that run succeeded. The deployment workflow is in turn triggered when CodeQL completes successfully. It also checks the CodeQL conclusion before running the deployment job.

The deployment job checks out the repository, installs dependencies, reruns the unit tests, authenticates to Azure, deploys the application package to App Service, imports the OpenAPI definition into APIM, and updates the APIM backend URL. This ordering makes the deployment dependent on successful CI and CodeQL completion, followed by a second test run immediately before deployment.

The workflows do not apply Terraform. Azure authentication, the named Web App, resource group, and APIM service must be configured in advance and must agree with the values expected by the workflow and infrastructure.

Infrastructure as Code

Terraform represents the desired Azure resources declaratively. The resource group is parameterized by variables for name and region; the remaining resource configuration specifies the service names, sizes, and Node runtime. Keeping these definitions in the repository makes the infrastructure easier to inspect and reproduce than ad hoc manual creation.

In the current implementation, infrastructure provisioning is a separate demonstration from application CI/CD. It is not automatically planned, applied, or drift-checked by the workflows.

Local demonstration

The PowerShell script `scripts/local-ci-cd.ps1` provides a sequential local demonstration of the lint, test, and build commands followed by Azure CLI commands for App Service deployment, APIM import, and backend configuration. It exits when an individual command reports failure. Running its cloud steps requires the Azure CLI, an authenticated Azure account, and the pre-provisioned resources.

5. Component interaction and lifecycle

1. A developer changes the application, OpenAPI contract, tests, or infrastructure definitions and pushes to `main` (or starts CI manually).
2. GitHub Actions checks out the repository and installs dependencies from `package-lock.json`.
3. Redocly and Spectral validate the OpenAPI file; Jest and Supertest exercise the Express route; the build command runs.
4. After successful CI, the CodeQL workflow analyzes the JavaScript source.
5. After successful CodeQL analysis, the deployment workflow reruns tests and authenticates to Azure using the configured secret.
6. The workflow deploys the application to the existing App Service.
7. The workflow imports the OpenAPI contract into the existing APIM service and points that API at the App Service backend.
8. Consumers can call the API through the APIM gateway, which forwards requests to the application.

The OpenAPI file therefore has two roles: it is checked during CI and is the source used to define the API imported into APIM. The application implements the corresponding route, while the deployment process connects the gateway definition to the application host.

6. Design decisions

API-first contract

Defining the API in OpenAPI provides an explicit interface independent of the Express implementation. This supports automated linting and gateway import, and makes the intended path, parameter, and response shape visible to developers and consumers. The trade-off is that a specification can drift from the implementation unless tests or contract checks compare them directly; the current test verifies the implementation response, but does not mechanically compare it with the OpenAPI schema.

Node.js and Express

Node.js and Express provide a small, familiar implementation for the single REST endpoint and work naturally with the Node-based CI tooling. Keeping the application deliberately small reduces setup overhead. The corresponding trade-off is that the API demonstrates little business logic, error handling, or integration with external data sources.

GitHub Actions

GitHub Actions keeps workflow definitions beside the application and uses repository events to automate the lifecycle. Splitting CI, CodeQL, and deployment into named workflows makes the stages independently visible in the Actions interface and establishes an explicit success sequence. The `workflow_run` chain is less direct than a single workflow with dependent jobs and requires careful configuration of workflow names, permissions, and secrets.

Terraform and Azure services

Terraform makes Azure resource configuration reviewable and repeatable. App Service provides a managed Node hosting target, while APIM demonstrates publication of the OpenAPI contract through a gateway. These managed services reduce the need to operate servers, but incur cloud cost and still require correct Azure identity, resource, and secret configuration.

Focused test scope

Jest and Supertest allow the route to be tested without deploying to Azure. This makes the test quick and deterministic. The trade-off is limited coverage: one happy-path response is tested, and the test does not exercise validation, error cases, APIM routing, or a live Azure deployment.

7. Limitations and trade-offs

- **Sample data only:** Temperature and condition are constants; there is no live weather source, persistence, or meaningful business logic.
- **One route and narrow test coverage:** Only a successful request is tested. The implementation does not currently provide explicit behavior for invalid cities or the `400` and `404` responses listed in the OpenAPI document.
- **Contract drift is possible:** The specification and application are linted/tested separately, without a contract test that proves the implementation conforms to every declared response.
- **Build verification is minimal:** `npm run build` is an echo command, so its success does not validate compilation or produce a deployment artifact.
- **Trigger coverage is limited:** CI runs on pushes to `main` and manual dispatch, not on pull requests. Changes on other branches do not receive automatic CI from this workflow.
- **Infrastructure is not part of automated delivery:** Terraform is not run by GitHub Actions. Resource provisioning, state management, drift detection, and teardown are outside the automated pipeline.
- **Single target environment:** There are no distinct development, staging, and production environments, deployment approvals, or rollback strategy.
- **Configuration consistency matters:** Terraform has fixed resource names, while deployment uses a Web App name secret and fixed APIM/resource-group values. These values must remain consistent; they are not wired together through Terraform outputs.
- **Credential-based Azure login:** Deployment relies on a long-lived credential stored in GitHub Secrets. Federated identity using OpenID Connect could reduce the need to maintain such credentials.
- **Gateway controls are not demonstrated:** The deployment imports and routes the API, but does not configure APIM policies, subscription-key enforcement, rate limits, or authorization. Those are possible APIM capabilities, not implemented protections in this project.
- **Operational concerns are out of scope:** The repository does not demonstrate monitoring, alerting, performance or load testing, dependency scanning in CI, or production incident handling.
- **Cloud cost and availability:** APIM and App Service require provisioned Azure resources and may incur cost. The workflow assumes those services exist and are available.

These limitations are appropriate for a focused educational demonstration, but they would need to be addressed before treating the pipeline as a production deployment process.

8. Conclusion

The project demonstrates a connected API lifecycle: an OpenAPI contract is linted, an Express implementation is tested, CI gates subsequent CodeQL analysis, and successful security analysis gates deployment to Azure App Service and API publication through APIM. Terraform describes the cloud resources, and a local PowerShell script illustrates the same checks and deployment operations.

The key design benefit is the separation of concerns while keeping the contract, application, infrastructure, and automation versioned together. The principal trade-off is scope: this is a compact educational proof of concept, not a complete production platform. In particular, pull-request validation, automated infrastructure provisioning, stronger contract and integration tests, managed identity, gateway policies, and multi-environment release controls would be valuable next steps.